Kingston University

Faculty of Science, Engineering and Computing

ME7761

MSc Mechatronic Systems

Project Report:

Trajectory Planning for ABB CRB 15000 Robot Using Bang-Coast-Bang Time Law

Author:

K2404618

Chirchir, K. Earl

Supervisor:

Dr. Claudio Gaz

January 29th , 2025

TABLE OF CONTENTS

LIST OF FIGURES:

ABBREVIATIONS:

ABB: Asea Brown Boveri (company name)

GOFA: Go-Faster (ABB's cobot series name)

IO: Input/Output

IP54: Ingress Protection Rating (54)

ISO: International Organization for Standardization

DH: Denavit-Hartenberg (a convention for robot kinematics)

DOF/DOFs: Degree(s) of Freedom

IK: Inverse Kinematics

TCP: Tool Center Point

CRB: Collaborative Robot

Abstract

This research presents the development and implementation of a trajectory planning system for the ABB CRB 15000 collaborative robot using the Bang-Coast-Bang time law. The study integrates three fundamental kinematic frameworks: direct kinematics for mapping joint configurations to end-effector poses through sequential transformation matrices, inverse kinematics for determining joint angles from desired end-effector positions using a gradient-based numerical solver, and inverse differential kinematics for computing joint velocities from desired end-effector velocities via the Jacobian pseudoinverse method.

The implementation begins with a comprehensive direct kinematics model based on Denavit-Hartenberg parameters, establishing the mathematical relationship between joint angles and end-effector pose through six successive coordinate transformations. This foundation enables accurate forward mapping of joint configurations to end-effector positions, incorporating precise geometric specifications including a base height of 265 mm, upper arm length of 444 mm, and end-effector length of 101 mm. The inverse kinematics solution employs a gradient descent approach with a learning rate of 0.01, iteratively minimizing positional error to determine joint configurations that achieve desired end-effector poses. This is complemented by an inverse differential kinematics implementation using the Moore-Penrose pseudoinverse of the Jacobian matrix, allowing for smooth velocity mapping between task space and joint space while handling kinematic redundancy.

The trajectory planning algorithm successfully implements a trapezoidal velocity profile, coordinating these kinematic relationships to achieve optimal motion between configurations while respecting velocity and acceleration constraints. The system maintains a convergence error threshold of 1e-6 across 10,000 iteration steps, ensuring accurate path following. Comprehensive testing demonstrated the algorithm's effectiveness through detailed analysis of joint angles, velocities, and end-effector trajectories. Results show successful coordination of all six joints, with velocities reaching up to 75 degrees per second while maintaining sub-centimeter tracking accuracy. The integration with RoboDK simulation software validated the practical applicability of the approach. This research contributes to the field of industrial robotics by providing an efficient solution for trajectory planning that balances time-optimal movement with practical constraints, suitable for various industrial applications.

Acknowledgement

I would like to express my heartfelt gratitude and appreciation to Dr. Claudio Gaz for his invaluable guidance, insights, and support, which enabled the successful completion of this project. His expertise and encouragement were instrumental in navigating challenges and ensuring the project's success. I would also like to appreciate the support received from family who have provided support during the entire journey.

1. BACKGROUND AND OBJECTIVES

1.1 Introduction

Robotic manipulators play a crucial role in contemporary industrial automation, enhancing productivity, precision, and effectiveness in production and logistics. The ABB CRB 15000 demonstrates the most recent progress in robotics technology. Nevertheless, in order to optimize the advantages of these sophisticated systems, it is important to employ advanced trajectory planning algorithms. These algorithms determine the robot's movement between configurations, considering factors such as velocity, acceleration, and route limitations.

Trajectory planning is a crucial concern in robotics, which entails the development of a chronological sequence of locations, velocities, and accelerations for every joint or end-effector of the robot. The efficacy of this planning is vital for the robot's performance, impacting aspects such as cycle time, energy consumption, and job execution quality. Optimizing trajectories in industrial contexts, where efficiency and reliability are crucial, may lead to substantial improvements in productivity and operational costs.

1.2 Background

The process of trajectory planning may be conceptually broken down into two primary components: route planning and time scaling. Path planning is the process of defining the precise path that a robot should take in either its joint space or task space. Time scaling gives a temporal evolution to the intended path, establishing how the robot moves along the path as a function of time.

The bang-coast-bang time rule, commonly known as a trapezoidal velocity profile, has gained popularity due to its simplicity and ability to provide time-optimal motion in various contexts. This profile consists of three separate phases:

1. Bang phase (acceleration): The robot accelerates at its highest rate till attaining a predefined maximum velocity.
2. Coast phase: The robot maintains a steady maximum velocity.
3. Bang phase (deceleration): The robot decelerates at its maximum pace until coming to a stop at the target place.

The ABB CBR 15000, the topic of this study, is an industrial robot built for payload handling and demanding manufacturing activities. It offers exceptional capabilities for a wide range of industrial applications. The robot offers 6 degrees of freedom, allowing it to reach any position and orientation within its workspace with remarkable flexibility.

1.3 Problem Statement

While the bang-coast-bang time law has proved useful for single-joint movements or simpler planar robots, its application to complicated 6-DOF industrial manipulators like the ABB CRB 15000 offers numerous obstacles. The fundamental issues stem from the necessity to coordinate

many joints to follow defined Cartesian routes while obeying different limitations.

Key challenges include:

1. **Kinematic mapping:** Translating desired Cartesian pathways into joint space movements while accounting for the robot's kinematic structure.
2. **Smoothness**: Generating trajectories that reduce jerk and vibration, which is critical for many industrial applications.
3. **Computational efficiency**: Developing algorithms that can create trajectories rapidly enough for real-time applications.

There is an obvious need for an integrated method that can solve these problems, exploiting the bang-coast-bang time rule to construct economical trajectories for the ABB CBR 15000 robot while maintaining smooth, viable, and near-optimal movements over arbitrary Cartesian pathways.

1.4 Objectives

The major aims of this project are:

1. Develop a kinematic trajectory planning system for the ABB CRB 15000 robot that employs a bang-coast-bang time law.
2. Implement the method in MATLAB and combine it with RoboDK for visualization and simulation.
3. Evaluate the performance of the algorithm in terms of trajectory smoothness and execution time.

1.5 Scope

To retain focus and accomplish significant outcomes within the project duration, the following scope is defined:

1. The study will concentrate on kinematic trajectory planning, assuming ideal dynamics and faultless trajectory tracking.
2. Path planning will not be handled directly. The method will use pre-defined Cartesian pathways as input.
3. Only point-to-point motions will be evaluated.
4. The simulation and assessment will be confined to the ABB CRB 15000 robot.
5. While the algorithm will be created with real-time applications in mind, the major focus will be on offline trajectory development and analysis.

1.6 Justification

Effective trajectory planning is vital for enhancing the performance and capabilities of modern industrial robots like the ABB CBR 15000. As industrial processes become increasingly complicated and demanding, the requirement for efficient, precise, and adaptable robot movements develops. The bang-coast-bang time rule presents a promising strategy for creating time-optimal trajectories, but its application to multi-joint robots following Cartesian pathways needs careful consideration of kinematic restrictions and practical limits.

This study fills a key gap in the present literature by establishing an integrated planning algorithm especially adapted to the ABB CRB 15000 robot. The resultant algorithm has the potential to improve the efficiency of robotic operations in industrial settings, leading to higher output and reduced cycle times.

The use of simulation tools like MATLAB and RoboDK allows for thorough testing and analysis without the requirement for actual hardware, permitting a full evaluation of the algorithm's performance over a wide range of circumstances. This technique permits quick iteration and improvement of the algorithm, ultimately leading to a more robust and effective solution.

2. STATE OF ART

1.7 Overview of Trajectory Planning

Trajectory planning for robotic manipulators has been a fundamental field of research in robotics for several decades. The objective is to construct a temporal history of locations, velocities, and accelerations for each joint or end-effector that allows the robot to accomplish a particular motion while fulfilling certain restrictions (LaValle, 2006). These restrictions may include joint limits, actuator limits, obstacle avoidance, and task-specific needs.
Siciliano et al. (2010) present a detailed overview of trajectory planning strategies, addressing both ancient and recent approaches. They underline the need to address both geometric and temporal components of motion planning.
Trajectory planning approaches may be roughly divided into joint space methods and Cartesian space methods. Each strategy has its advantages and limitations, which have been widely researched in the literature by (Gasparetto et al., 2015).

1.8 Joint Space Trajectory Planning

Joint space trajectory planning includes directly describing the motion of each joint across time. This technique has the benefit of operating in the robot's native coordinates and readily managing joint constraints. Common ways include:

1. Polynomial interpolation: Fitting polynomial functions to fulfill boundary requirements on location, velocity, and acceleration. Craig (2009) describes the use of cubic and higher-order polynomials for smooth joint trajectories.

2. Spline interpolation: Using piecewise polynomial functions to generate smoother trajectories. Biagiotti and Melchiorri (2009) give a detailed review of spline-based approaches for robotic trajectory planning.

3. Optimal control approaches: Formulating trajectory planning as an optimization problem to reduce criteria like execution time or energy usage. Von Stryk and Bulirsch (1992) give a foundational study on direct and indirect approaches for trajectory optimization.
Lin et al. (1983) offer a time-optimal trajectory planning approach in joint space that includes both kinematic and dynamic constraints. Their solution employs a convex optimization concept to construct smooth and efficient trajectories.

The fundamental problem with joint space planning is that the resulting end-effector motion in Cartesian space can be unintuitive and could lead to collisions or unwanted routes. To address this, hybrid techniques that include both joint and Cartesian space limitations have been proposed (Katzschmann et al., 2019).

1.9 Cartesian Space Trajectory Planning

Cartesian space planning describes the required end-effector motion directly in task space coordinates. This enables for more straightforward path selection and simpler obstacle avoidance. Approaches include:

1. Linear segments with parabolic blends: Combining straight-line movements with smooth transitions. Taylor (1979) established this system, which remains popular in industrial robots because to its simplicity and predictability.
2. Parametric curves: Using functions like Bezier curves or B-splines to express complicated pathways. Elbanhawi and Simic (2014) present a detailed analysis of sampling-based motion planning strategies, including the use of parametric curves.
3. Potential field methods: Generating trajectories by following the gradient of an artificial potential field. Khatib's (1986) important work on real-time obstacle avoidance established the framework for prospective field techniques in robots.

Gasparetto and Zanotto (2010) give a review of trajectory planning strategies particularly for industrial robotics, including both standard and sophisticated approaches in Cartesian space.

The major problem in Cartesian space planning is mapping the Cartesian trajectory to joint space motion while obeying the robot's kinematic limitations and avoiding singularities. Chettibi et al. (2004) describe a technique for optimum trajectory planning in Cartesian space that takes into consideration dynamic limitations and singularity avoidance.

1.10 Time Scaling and the Bang-Coast-Bang Approach

Time scaling entails assigning a temporal evolution to a geometric route. The bang-coast-bang profile, also known as a trapezoidal velocity profile, is a commonly utilized method due to its simplicity and optimality qualities.

Pfeiffer and Johanni (1987) give a basic study of the bang-bang concept for time-optimal trajectory planning. They show that for a single joint with velocity and acceleration constraints, this profile provides the time-optimal motion between two places.

However, coordinating many joints to follow a specific route brings extra complexity. Bobrow et al. (1985) and Bobrow et al. (1985) separately established influential approaches for time-optimal trajectory planning along defined pathways subject to torque limitations. These algorithms solve for the ideal velocity profile along the path, which commonly results in a bang-bang or bang-singular-bang control structure.

Recent work has focused on efficient implementations and expansions of these concepts:

- Pham (2014) provided a method for smooth time-optimal path parameterization that eliminates discontinuities in the control inputs.
- Verscheure et al. (2009) framed the issue as a convex optimization, enabling for efficient numerical solution.
- Kunz and Stilman (2015) proposed a probabilistically complete approach for time-optimal route parameterization with dynamic constraints.

1.11 Trajectory Planning for Industrial Robots

Industrial robots like the ABB CRB 15000 have special concerns that affect trajectory planning:

1. Large workplaces with potential singularities
2. High payload capacity necessitating consideration of dynamic effects
3. Need for collaboration with other systems and processes
4. Safety criteria and collision avoidance

Biagiotti and Melchiorri (2009) introduced a trajectory planning system particularly built for industrial robots, concentrating on providing smooth movements with restricted jerk. Their technique employs cubic splines and can handle both point-to-point and continuous path following jobs. Lange and Albu-Schäffer (2016) offers a method for online trajectory creation with kinematic and dynamic limitations, which is particularly useful for industrial applications needing real-time flexibility. Singularity handling in trajectory planning for industrial robots has been addressed by numerous researchers. Kojima and Kibe (2013) offers a method for singularity-robust trajectory planning utilizing null space motion. Patel and George (2012) present a detailed overview of manipulator singularities and solutions for their avoidance in trajectory planning.

1.12 Simulation and Evaluation

Simulation plays a significant role in designing and assessing trajectory planning algorithms. MATLAB is commonly used for designing and implementing planning methods due to its robust numerical computation capabilities and robotics toolboxes. Corke's Robotics Toolbox includes a complete set of functions for robot kinematics, dynamics, and trajectory development in MATLAB.
Evaluation metrics for trajectory planning algorithms commonly include:
1. Execution time
2. Smoothness (typically assessed by jerk)
3. Tracking accuracy
4. Constraint fulfillment (joint limits, singularity avoidance, etc.)
5. Computational efficiency
Cetin et al. (2017) give a comparative assessment of alternative trajectory planning algorithms for industrial robots, giving insights into evaluation procedures and performance measures.

1.13 Gap Analysis

While extensive research has been undertaken on trajectory planning for robotic manipulators, numerous gaps exist in the context of applying bang-coast-bang time scaling to industrial robots like the ABB CRB 15000:

1. Most existing work on bang-coast-bang profiles focuses on single-joint motion or simpler planar robots. Extension to 6-DOF industrial manipulators following arbitrary Cartesian trajectories is not well-explored.
2. The interplay between bang-coast-bang time scaling and the kinematic limitations of redundant robots (such singularity avoidance) needs additional exploration.

3. Practical factors for industrial applications, such as smooth start/stop motions and cooperation with other processes, are typically disregarded in theoretical approaches.
4. Integration of trajectory planning methods with contemporary simulation tools like RoboDK for industrial robots is not well-documented in the literature.

This research intends to solve these shortcomings by designing an integrated trajectory planning technique designed for the ABB CRB 15000 robot, leveraging bang-coast-bang time scaling, and testing it through simulation.

3. METHODOLOGY

To achieve the project objectives, the following methodology will be employed:

3.1 Description of the ABB CRB Robot

The ABB CRB 15000 is an advanced collaborative robot (cobot) designed by ABB for safe human-robot interaction in industrial settings. This innovative robot combines power and safety, capable of handling payloads up to 5kg while featuring integrated torque sensors in all six joints that enable immediate stopping if contact with a human worker is detected. The robot's design eliminates the need for protective barriers or fencing, allowing it to work directly alongside humans in shared workspaces.

Figure 1 The ABB CRB 15000 collaborative robot.

With a reach of 950mm, which is 12% longer than comparable 5kg cobots in its class, the robot demonstrates impressive performance capabilities. It achieves a maximum Tool Center Point (TCP) speed of 2.2 m/s, though the safe collaborative speed is lower and should be configured through the SafeMove configurator App. The robot's precision is evident in its pose repeatability of 0.05mm, and it can complete a 1kg picking cycle (25 × 300 × 25mm) in just 0.66 seconds.

The robot’s construction is relatively lightweight at 27kg, with a compact base dimension of 165 × 165mm. It offers remarkable flexibility in terms of mounting options, as it can be installed at any angle, including table, wall, and ceiling mounting. The robot features six axes of movement, with working ranges of ±180° for most joints, except for axis 3 which ranges from -225° to 85°. The maximum axis speeds vary from 125°/s for the base rotation and arm to 200°/s for the wrist, bend, and turn movements.

In terms of technical specifications, the robot operates with the OmniCore C30 controller and includes an IP54 protection rating. It comes equipped with a standard ISO 9409-1-50 tool flange and provides customer power supply (24V/1.5A) along with four signals for IO, Fieldbus, or Ethernet connectivity. The robot's safety features are certified to Category 3, PL d, and it includes SafeMove Collaborative functionality as standard.

The robot is versatile in its applications, being suitable for material handling, machine tending, assembly, picking and packaging, and screwdriving tasks. Its ease of use is enhanced through intuitive graphical Apps on the FlexPendant, featuring lead-through programming capabilities and enhanced interaction through the arm side interface. This combination of safety, speed, and user-friendly operation makes the GoFa an efficient solution for various industrial automation needs.

3.2 Kinematic Analysis

3.2.1 Direct Kinematics

Direct kinematics is a fundamental concept in robotics that deals with determining the position and orientation of a robot's end-effector given the joint angles or displacements of its links. This process is essential for understanding and controlling the motion of robotic manipulators in various applications, from industrial automation to medical robotics.

The direct kinematics problem involves mapping the joint space of a robot to its task space, which represents the Cartesian coordinates and orientation of the end-effector. This mapping is typically achieved through a series of coordinate transformations that relate the positions of successive links in the kinematic chain. The most common method for representing these transformations is the Denavit-Hartenberg (D-H) convention, introduced by Jacques Denavit and Richard S. Hartenberg in 1955 (Denavit and Hartenberg, 1955; Spong et al., 2006).

The D-H convention provides a systematic approach to assigning coordinate frames to the links of a robotic manipulator. For each link, four parameters are defined: the link length (a), the link twist (α), the link offset (d), and the joint angle (θ). These parameters are used to construct homogeneous transformation matrices that describe the relationship between adjacent link coordinate frames. The overall transformation from the base frame to the end-effector frame is then obtained by multiplying these individual transformation matrices in sequence (Spong et al., 2006).

The general form of a D-H transformation matrix is given by:

$$T_{i-1,i} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i \cos\alpha_i & \sin\theta_i \sin\alpha_i & a_i \cos\theta_i \\ \sin\theta_i & \cos\theta_i \cos\alpha_i & -\cos\theta_i \sin\alpha_i & a_i \sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This matrix represents the transformation from frame i-1 to frame i. By multiplying these matrices for all the links in the kinematic chain, we obtain the overall transformation matrix:

$$T_{0,n} = T_{0,1} \cdot T_{1,2} \cdot \ldots \cdot T_{n-1,n}$$

The resulting matrix contains the position and orientation information of the end-effector with respect to the base frame. The upper-left 3x3 submatrix represents the rotation, while the first three elements of the fourth column represent the position vector (Siciliano et al., 2010).

3.2.2 Denavit-Hartenberg Formulation Analysis

3.2.2.1 Overview

This technical analysis presents a detailed examination of the Denavit-Hartenberg (DH) formulation for the ABB CRB 15000 GOFA 5 collaborative robot. The robot features six revolute joints with significant geometric complexities, including elbow and wrist offsets. This analysis covers the systematic frame assignment, parameter derivation, and kinematic implications of the chosen DH convention implementation.

The DH convention was established in the year 1955 by Jacques Denavit and Richard S. Hartenberg. Their work provided the need for a common natural methodology for kinematics analysis, due to the enhanced complicated nature of the industrial robots, multi-degree-of-freedom robotic manipulators. The convention has since then developed into standard practice in robots research and application, being the foundation of many current control and trajectory planning techniques.

To derive the kinematics of a robotic arm, the DH convention establishes a series of coordinate frames along the links of the manipulator. These frames are related using transformation matrices constructed from four parameters: $a_i, \alpha_i, d_i,$ and θ_i.

Each link is described by these parameters:

1. **Link Length (a_i):** The distance between the z_{i-1} and z_i axes along the common normal (measured along the x_{i-1}-axis).
2. **Link Twist (α_i):** The angle between z_{i-1} and z_i axes, measured about the x_{i-1}-axis.
3. **Link Offset (d_i):** The distance along the z_i-axis to the common normal (measured along the x_{i-1}-axis).
4. **Joint Angle (θ_i):** The angle of rotation around the z_i-axis from x_{i-1} to x_i.

The kinematic modeling of the ABB CRB 15000 GOFA 5 robot employs the Denavit-Hartenberg (DH) convention, a systematic methodology for describing the geometric relationships between consecutive joint frames in a robotic manipulator. This approach utilizes four fundamental parameters for each joint-link combination: the joint angle (θ), link offset (d), link length (a), and link twist (α). These parameters collectively provide a complete geometric description of the robot's kinematic chain.

3.2.2.2 Frame Assignment Analysis

The first step in deriving the kinematic model of a manipulator using the DH convention involves defining the appropriate coordinate frames for the manipulator.

As described by Spong et al. (2006), a robotic manipulator consists of a sequence of joints and links. The joints facilitate rotational movement around their respective axes. To represent the relative motion between these components, frame transformations are employed. For this purpose, each link must have an associated coordinate frame, referred to as a "link frame." While frames can be assigned in various ways, adhering to the DH convention requires following a specific set of rules.

The initial frame, often termed the base frame, serves as the reference frame and is typically positioned at the base of the mechanism. Subsequent frames, labeled i, are constructed based on the following principles outlined by Spong et al. (2006):

- The Z_i-axis is aligned in the positive direction of the joint rotation, following the right-hand rule (RHR). According to the RHR, if the thumb of the right hand points along the line of action, the curled fingers indicate the positive axis direction.
- The X_i-axis is defined along the common normal to Z_i and Z_{i-1}, pointing away from the previous frame. If $Z_i$and Z_{i-1} are parallel, there are infinite common normals. In this case, the X_i-axis is positioned arbitrarily as long as it remains perpendicular to both Z_i and Z_{i-1}.
- The Y_i-axis is determined to complete the frame according to the RHR. Specifically, if the right thumb points along the X-axis, the index finger along the Y-axis, and the middle finger along the Z-axis, the frame orientation is consistent with the RHR.

Once these rules are applied, determining the frame origin becomes straightforward.

The frame assignment for the ABB CRB 15000 robot adheres to the standard Denavit-Hartenberg (DH) convention, with adjustments tailored to the robot's geometry. The base frame, or Frame 0, is defined with its origin positioned on the floor. The Z_0 axis is aligned with the rotation axis of the first joint, while the X_0 axis is selected to reduce the number of non-zero parameters in the DH parameters. Intermediate frames (Frames 1 through 5) are assigned according to the DH convention rules, ensuring that the Z axes are aligned with the respective joint rotation axes. Particular attention is given to specific geometric features, such as the elbow offset at Frame 3 and the wrist offset at Frame 5, with the X axes positioned along the common normals between successive joint axes. The end-effector frame, or Frame 6, is defined with its origin at the center of the final flange, facilitating the mounting of end-effector tools. Its Z_6 axis is aligned with the approach direction to streamline operations.

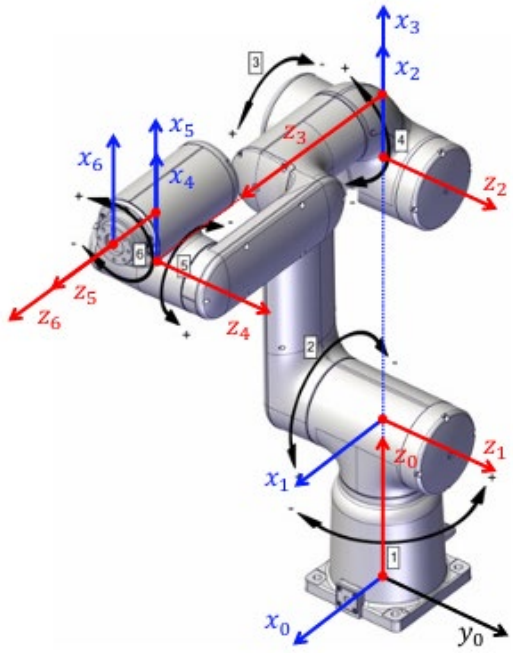


Figure 2 Assignment of DH frames for the ABB CRB 15000 robot

3.2.2.3 DH Parameter Table and Analysis

The complete DH parameter table for the ABB CRB 15000 is as follows:

Joint (i)	α_i (rad)	a_i (mm)	d_i (mm)	θ_i (rad)
1	$-\pi/2$	0	265	q_1
2	0	444	0	q_2
3	$-\pi/2$	110	0	q_3
4	$\pi/2$	0	470	q_4
5	$-\pi/2$	80	0	q_5
6	0	0	101	q_6

Table 1 DH parameter table for the ABB CRB 15000

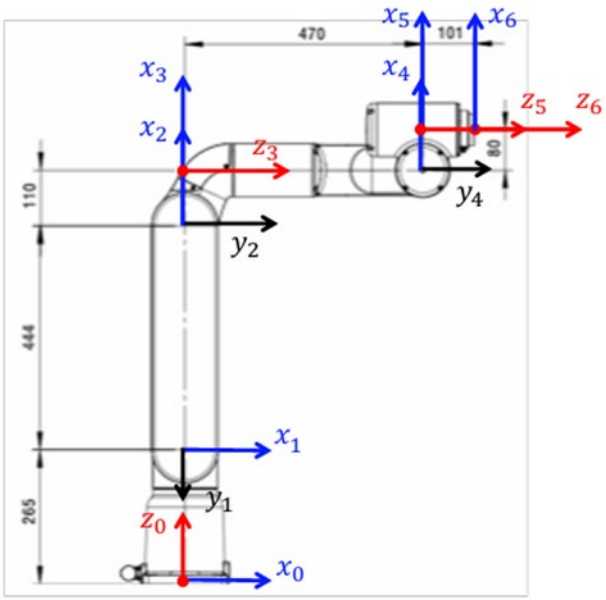


Figure 3 Another view of the DH frames assigned

The parameter analysis of the ABB CRB 15000 robot provides insight into the structural and geometric design of its manipulator. The link twist angles (α_i) follow an alternating pattern of $\pm\pi/2$, reflecting the perpendicular arrangement of successive joint axes. Zero values for α_i occur where joint axes are parallel, specifically between joints 2-3 and 5-6. The link lengths (a_i) highlight key structural dimensions, with significant values at $a_2 = 444$ mm and $a_3 = 110$ mm representing the main arm segments. A smaller offset at $a_5 = 80$ mm accounts for the wrist mechanism, while zero values indicate collinear joint axes.

Joint offsets (d_i) also play a critical role in the robot's geometry. The base height, $d_1 = 265$ mm, elevates the manipulator above the ground, while the major vertical offset, $d_4 = 470$ mm, defines the upper arm structure. The end-effector offset, $d_6 = 101$ mm, establishes the position of the tool mounting point. Joint angles (θ_i) are all variable and represented as q_i. In the reference position, these angles are defined as $q_1 = 0$, $q_2 = -\pi/2$, $q_3 = 0$, $q_4 = 0$, $q_5 = 0$, and $q_6 = 0$. This configuration provides a baseline for analyzing the robot's kinematics and motion.

These parameters make the forward kinematic calculation easier because they identify the position and orientation of the manipulator's end-effector with respect to joint angles and link lengths. By successively multiplying the transformation matrices of each link, the position and orientation of the end-effector can be obtained relative to the base frame. This systematic approach reduces computational complexity, making it ideal for both analysis and real-time applications.

With regard to the ABB CRB robot, the individual transformation matrices that were multiplied included:

$$ {}^{0}_{1}T = \begin{bmatrix} cq_1 & 0 & -sq1 & 0 \\ sq_1 & 0 & cq_1 & 0 \\ 0 & -1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{1}_{2}T = \begin{bmatrix} cq_2 & -sq_2 & 0 & a2 * cq2 \\ sq_2 & cq_2 & 0 & a2 * sq2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{2}_{3}T = \begin{bmatrix} cq_3 & 0 & -sq3 & a3 * cq3 \\ sq_3 & 0 & cq_3 & a3 * sq3 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{3}_{4}T = \begin{bmatrix} cq_4 & 0 & sq_4 & 0 \\ sq_4 & 0 & -cq_4 & 0 \\ 0 & 1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{4}_{5}T = \begin{bmatrix} cq_5 & 0 & -sq_5 & a5 * cq5 \\ sq_5 & 0 & cq_5 & a5 * sq5 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{5}_{6}T = \begin{bmatrix} cq_6 & -sq6 & 0 & 0 \\ cq_6 & cq6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

Due to the size of the homogeneous transformation matrix obtained, the resultant calculation was done in MATLAB. The relationship between the manipulator's individual frames allows the connection between joint angles and cartesian positions to be established through the process of matrix multiplication. This process yields a resulting matrix, commonly referred to as Forward Kinematics or Direct Kinematics.

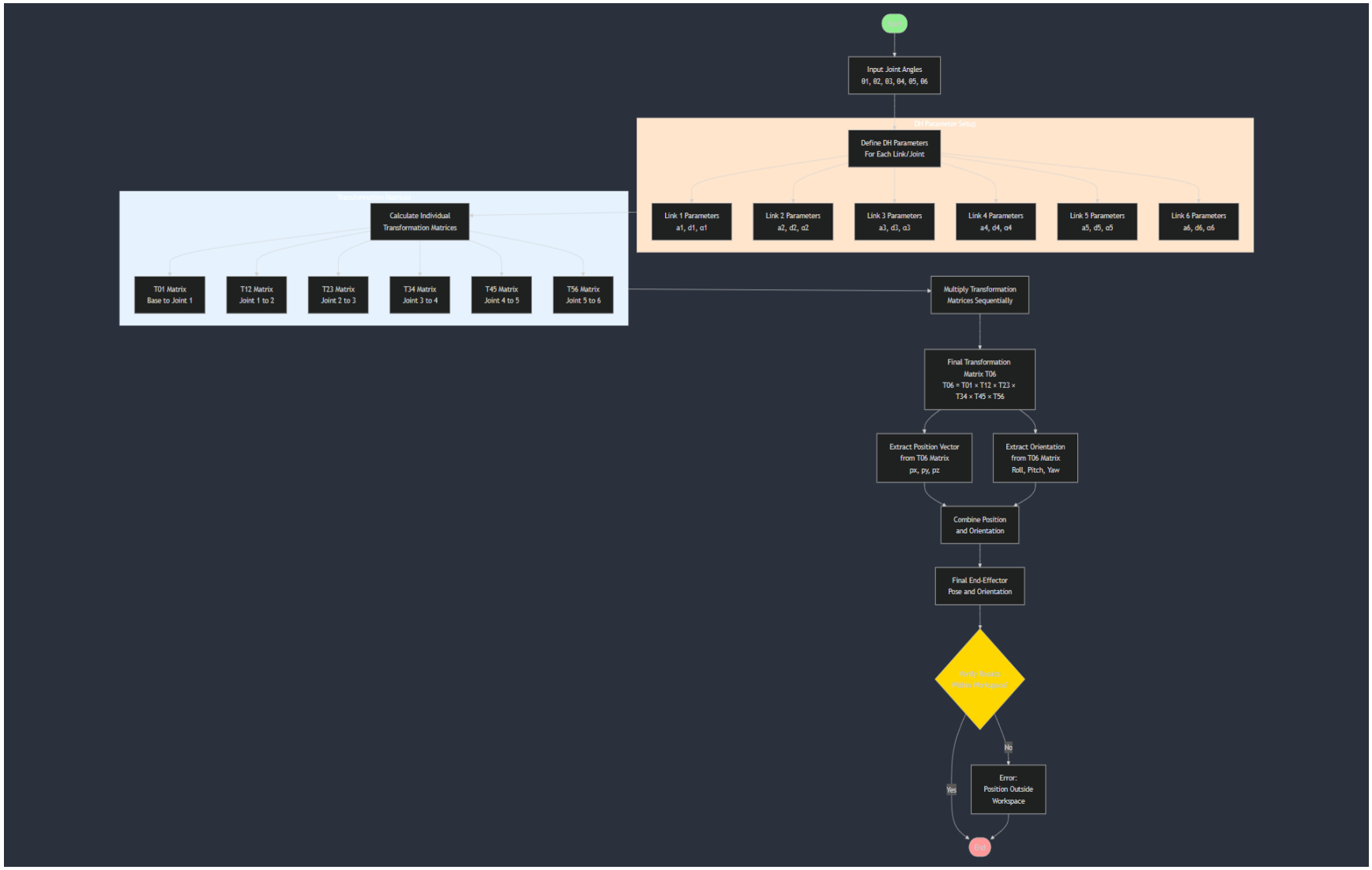


Figure 4 Direct Kinematics Flowchart

3.3 Inverse Kinematics

Inverse kinematics (IK) is a fundamental concept in robotics that plays a crucial role in the control and manipulation of robotic systems. It involves determining the joint configurations of a robot that will achieve a desired end-effector position and orientation. This process is essential for enabling robots to interact with their environment and perform complex tasks with precision and accuracy.

The inverse kinematics problem is inherently more challenging than its counterpart, forward kinematics, due to its non-linear nature and the potential for multiple solutions or no solutions at all. In forward kinematics, the end-effector position and orientation are calculated based on known joint angles, resulting in a straightforward, one-to-one mapping. Conversely, inverse kinematics seeks to find the joint angles that will produce a specific end-effector pose, which can lead to multiple possible configurations or scenarios where no valid solution exists (Craig, 2009).

The mathematical formulation of the inverse kinematics problem can be expressed as follows: Given a desired end-effector pose $X_d \in SE(3)$, find a set of joint angles θ that satisfy the equation $F(\theta) = X_d$, where $F(\theta)$ is typically a highly non-linear function involving trigonometric relationships between joint angles and link lengths (Siciliano et al., 2010).

Numerical methods employ iterative algorithms to approximate solutions to the inverse kinematics problem. These methods are more versatile and can be applied to a wide range of robot configurations, including redundant manipulators. One commonly used numerical approach is the Jacobian-based method, which utilizes the Jacobian matrix to iteratively refine joint angle estimates (Buss, 2004). This the method that was implemented in the ABB CRB 15000 robot.

The Jacobian matrix $J(\theta)$ represents the linear transformation between joint velocities and end-effector velocities. It is defined as:

$$J(\theta) = \frac{\partial F(\theta)}{\partial \theta}$$

Where $F(\theta)$ is the forward kinematics function. The Jacobian inverse method uses this matrix to compute joint angle updates iteratively:

$$\Delta\theta = J^{-1}(\theta)\Delta X$$

Where ΔX is the difference between the desired and current end-effector pose. This process is repeated until the end-effector reaches the desired pose within a specified tolerance (Sciavicco and Siciliano, 2000).

However, the Jacobian inverse method can encounter difficulties when the Jacobian matrix becomes singular or ill-conditioned, which occurs at kinematic singularities. To address this issue, various techniques have been developed, such as the damped least squares method and the Jacobian transpose method (Wampler, 1986).

Although the forward kinematics equations link the end-effector's position to the joint angles, they don't directly reveal the end-effector's velocities or how they relate to the joints' velocities. This section is essential for determining a manipulator's inverse kinematics because it establishes the possibility of iteratively adjusting the joint angles until the desired position and orientation

are attained by connecting the end-effector's velocities to the joint velocities. This relationship can be roughly described using the Jacobian.

A mathematical concept called the Jacobian is used to examine the relationship between two sets of variables. It specifically shows a vector-valued function's derivative with respect to input variables. The Jacobian matrix, for instance, explains how the elements of the output vector vary in relation to the elements of the input vector when a function takes multiple variables as inputs and outputs a vector. This implies that every Jacobian element can be viewed as a partial derivative.

Therefore, the relationship between the end-effector and joint velocities can be found using the Jacobian. The Jacobian matrix explains how slight variations in the joint angles translate into adjustments to the end-effector's position and orientation by taking the partial derivatives of the forward kinematics equations with regard to the joint angles.

It is crucial to recognise that the previously derived kinematic relations are non-linear in order to comprehend the relationship between the Jacobian and numerically solving the inverse kinematics. Therefore, determining the proper joint-angle configuration from a particular end-effector pose basically entails solving a non-linear optimisation problem.

Numerical techniques have long been used to solve non-linear optimisation problems. Although there are several numerical techniques, most of them rely on Taylor expansions to obtain the first or second-order system shown in the following equation.

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2}(x-a)^2 + \mathcal{O}((x-a)^3)$$

An approximation of a non-linear function can be obtained by employing the Taylor expansion of the function. This will result in the accurate representation of the function. Utilising the first-order system,

$$f(x) = f(a) + f'(a)(x-a) + \mathcal{O}((x-a)^2)$$

Taking into account the fact that this is a non-scalar context and having the appropriate notation, the Taylor expansion would look as follows:

$$f(\theta_d) = f(\theta_i) + \left(\frac{\partial f(\theta)}{\partial \theta}\right)\Big|_{\theta_i} (\theta_d - \theta_i) + \mathcal{O}((\theta_d - \theta_i)^2)$$

Where θ_d indicates the desired state while θ_i denotes the current state which the linearization is made around. Solving for $(\theta_d - \theta_i)$ while excluding the higher order terms one will end up with:

$$(\theta_d - \theta_i) = \left(\frac{\partial f(\theta)}{\partial \theta}\right)^{-1}\Big|_{\theta_i} \left(f(\theta_d) - f(\theta_i)\right)$$

And simplifying the notation yields the equation below where the Jacobian Inverse presents itself as the relation between the joint angles to the end effector pose.

$$\Delta\theta = J^{-1}(\theta)\Delta X$$

In the equations, ΔX is the error vector for the desired pose and current pose, while the Jacobian represents the following matrix:

$$J = \begin{bmatrix} \frac{\partial x}{\partial \theta_1} & \cdots & \frac{\partial x}{\partial \theta_n} \\ \frac{\partial y}{\partial \theta_1} & \cdots & \frac{\partial y}{\partial \theta_n} \\ \frac{\partial z}{\partial \theta_1} & \cdots & \frac{\partial z}{\partial \theta_n} \\ \frac{\partial \theta_z}{\partial \theta_1} & \cdots & \frac{\partial \theta_z x}{\partial \theta_n} \\ \frac{\partial \theta_y}{\partial \theta_1} & \cdots & \frac{\partial \theta_y}{\partial \theta_n} \\ \frac{\partial \theta_z}{\partial \theta_1} & \cdots & \frac{\partial \theta_z}{\partial \theta_n} \end{bmatrix}$$

Which can be derived in the following manner:

$$J = \begin{bmatrix} R_{i-1}\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \times (d_n - d_{i-1}) \\ R_{i-1}\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \end{bmatrix}$$

Here, R_{i-1} denotes the rotation matrix, while d_{i-1} and d_n represent the current position and the final position of the end effector, respectively.

To perform linearization, it is necessary to take the inverse of the Jacobian. However, this process can encounter challenges related to numerical stability, particularly when the Jacobian is non-invertible. This issue arises in scenarios where the Jacobian, which maps the relationship between Cartesian and joint velocities of a robotic manipulator, has a rank lower than the system's degrees of freedom (rankJ < #DOFs). Such a condition is referred to as a singularity.

3.4 Dependence on the Jacobian Inverse

As previously discussed, the Jacobian provides a means to connect the velocities in joint space to those in Cartesian space. However, the issue is more complex because the manipulator can encounter singular configurations in various regions of its valid workspace. These singular configurations cause the Jacobian inverse to become numerically unstable, making it necessary to approximate the inverse to achieve a feasible solution using numerical methods. Two commonly employed approaches for approximating the Jacobian inverse are the Moore-Penrose pseudo-inverse and the Jacobian Transpose.

- Jacobian Pseudo Inverse

$$\Delta\theta = J^{\dagger}\Delta X$$

- Jacobian Transpose

$$\Delta\theta = J^{T}\Delta X$$

3.5 Gradient Descent

The gradient descent method is a numerical technique used to solve a set of n non-linear algebraic equations with n unknowns. It was notably applied to address the inverse kinematics problem by both Balestrio (1984) and Wolovich (1984). This method is rooted in the foundational optimization algorithm known as gradient descent but incorporates some ingenious modifications to adapt it to this context.

The gradient descent method only makes use of the first order derivative of the Taylor expansion and can, in general, be formulated according to the equation below, where n is the *n-th* iteration, x is the optimization variable, $\nabla f(x_n)$ the gradient of the optimization function and α the learning rate (which is a tune-able design parameter for the gradient descent algorithm).

In essence, the gradient descent method relies solely on the first-order derivative of the Taylor expansion and can generally be expressed using the equation below. Here, n represents the *n-th* iteration, x is the optimization variable, $\nabla f(x_n)$ denotes the gradient of the optimization function, and α is the learning rate, a tunable parameter in the gradient descent algorithm:

$$x_{n+1} = x_n - \alpha\nabla f(x_n)$$

To compute the gradient, the method employs the Jacobian transpose, J^T, along with the residual, r(x), as shown in the following equation:

$$x_{n+1} = x_n - \alpha J^T r(x)$$

This approach leverages the property that the Jacobian transpose results in a matrix whose columns correspond to the gradient vectors of each residual with respect to the parameters. By multiplying the Jacobian transpose by the residual vector, the result is the gradient vector.

The gradient descent method solves the inverse kinematics problem by iteratively updating the joint angles to reduce the error between the desired and actual positions of the end effector. This iterative adjustment process is visually represented in the figure below, using the example of a single joint. Specifically, the method utilizes the Jacobian transpose to map the positional error between the actual and desired end-effector positions into the joint-angle space. In the context of inverse kinematics, x_n and x_{n+1} represent joint angles, while the residual r(x) corresponds to the error vector between the current pose and the target pose.

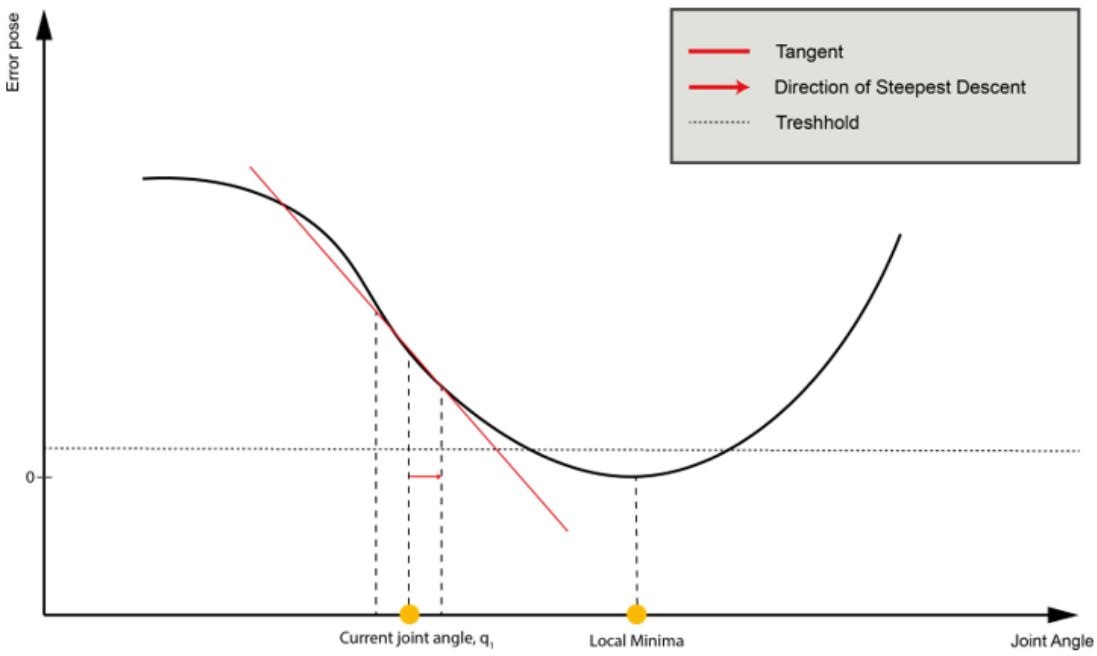


Figure 5 Gradient Descent Illustration

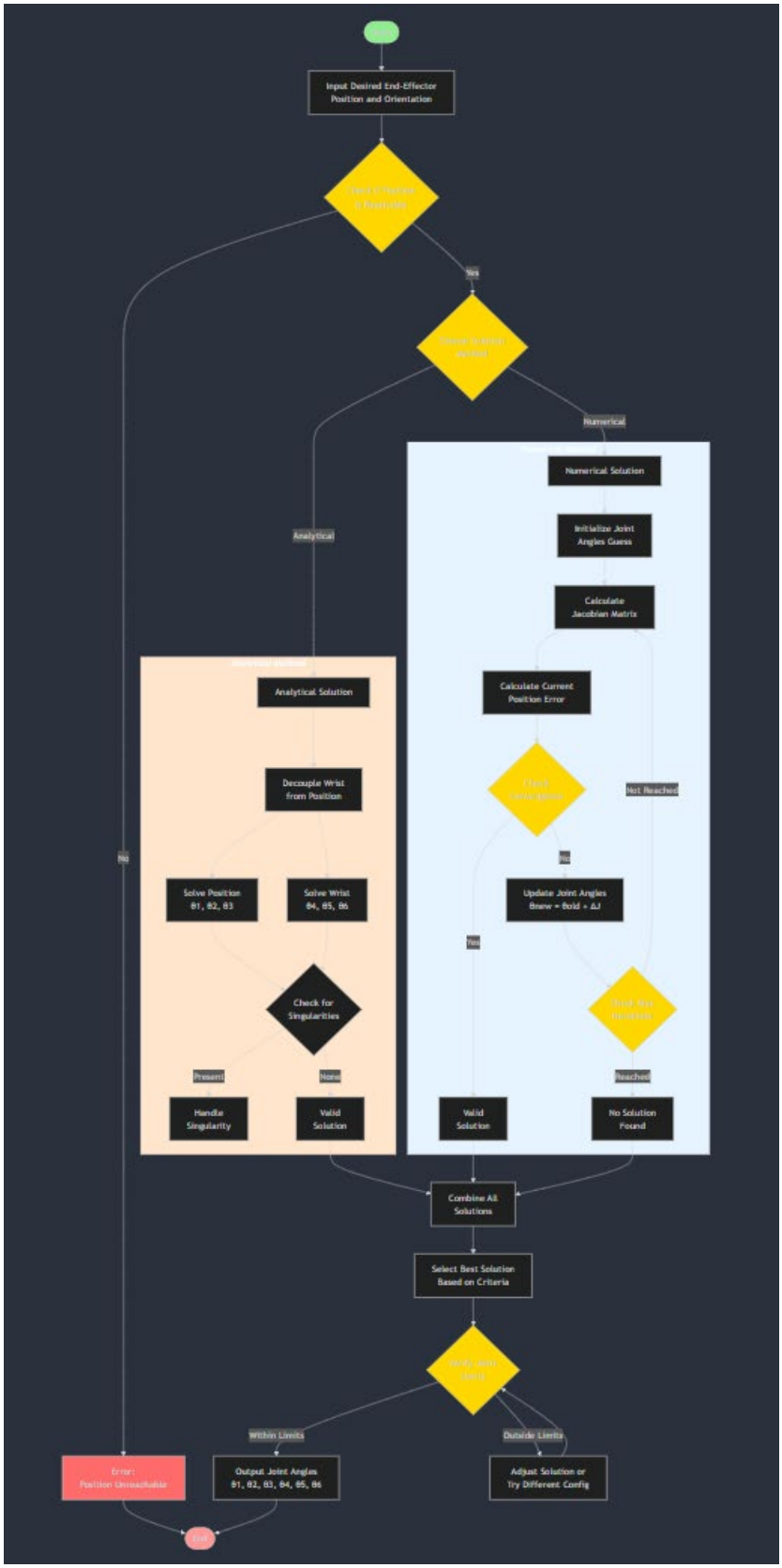


Figure 6 Inverse Kinematics Flowchart

3.6 Inverse Differential Kinematics

Inverse differential kinematics is a key concept in the field of robotics, focused on calculating the joint velocities necessary to produce a specified velocity of the end-effector in Cartesian space. This methodology plays a vital role in robot motion planning and control, enabling robots to perform smooth, accurate, and efficient movements within their working environment. The core principle of inverse differential kinematics is based on the mathematical relationship between the velocities of the robot's joints and its end-effector, which is governed by the Jacobian matrix. The Jacobian matrix, denoted as J(q), is a configuration-dependent transformation that links joint velocities to end-effector velocities through the equation:

$$\dot{x} = J(q)\dot{q}$$

Where $\dot{x}$ represents the end-effector velocity vector, $\dot{q}$ represents the joint velocity vector, and q denotes the current joint configuration. The Jacobian matrix encapsulates the instantaneous kinematics of the robot, providing a linear approximation of the robot's motion at a given configuration (Siciliano et al., 2010).

The inverse differential kinematics problem involves solving for the joint velocities given a desired end-effector velocity. In its simplest form, this can be expressed as:

$$\dot{q} = J^{-1}(q)\dot{x}$$

However, this straightforward inversion is only applicable when the Jacobian is square and non-singular. In practice, robotic systems often have redundant degrees of freedom or encounter singular configurations, necessitating more sophisticated approaches (Spong et al., 2006).

One common method for solving the inverse differential kinematics problem is the use of the Moore-Penrose pseudoinverse, denoted as J^{+}. This approach provides a least-squares solution that minimizes the norm of the joint velocities:

$$\dot{q} = J^{+}(q)\dot{x}$$

The pseudoinverse method is particularly useful for redundant robots, as it allows for the incorporation of secondary objectives in the null space of the Jacobian. This can be expressed as:

$$\dot{q} = J^{+}(q)\dot{x} + (I - J^{+}(q)J(q))\dot{q}_0$$

Where $\dot{q}_0$ represents an arbitrary joint velocity vector that is projected onto the null space of J, allowing for additional criteria to be optimized without affecting the primary task (Escande et al., 2014). This is the method that was implemented to the ABB CRB 15000 robot in MATLAB.

In scenarios where singularities are a concern, damped least squares methods are often employed. These techniques introduce a damping factor λ to regularize the solution:

$$\dot{q} = J^{T}(JJ^{T} + \lambda^{2}I)^{-1}\dot{x}$$

This formulation improves the numerical stability of the inverse kinematics solution near singular configurations, at the cost of introducing small errors in the end-effector velocity tracking (Buss, 2004).

For more complex robotic systems, particularly those with a high number of degrees of freedom or subject to multiple constraints, optimization-based approaches have gained popularity. These methods formulate the inverse differential kinematics problem as a quadratic programming (QP) problem:

$$\min_{\dot{q}} \frac{1}{2} \dot{q}^T H \dot{q} + f^T \dot{q}$$
subject to $A\dot{q} \leq b$

where H and f define the objective function, and A and b represent inequality constraints. This formulation allows for the incorporation of joint limits, obstacle avoidance, and other constraints directly into the inverse kinematics solution (Wampler, 1986).

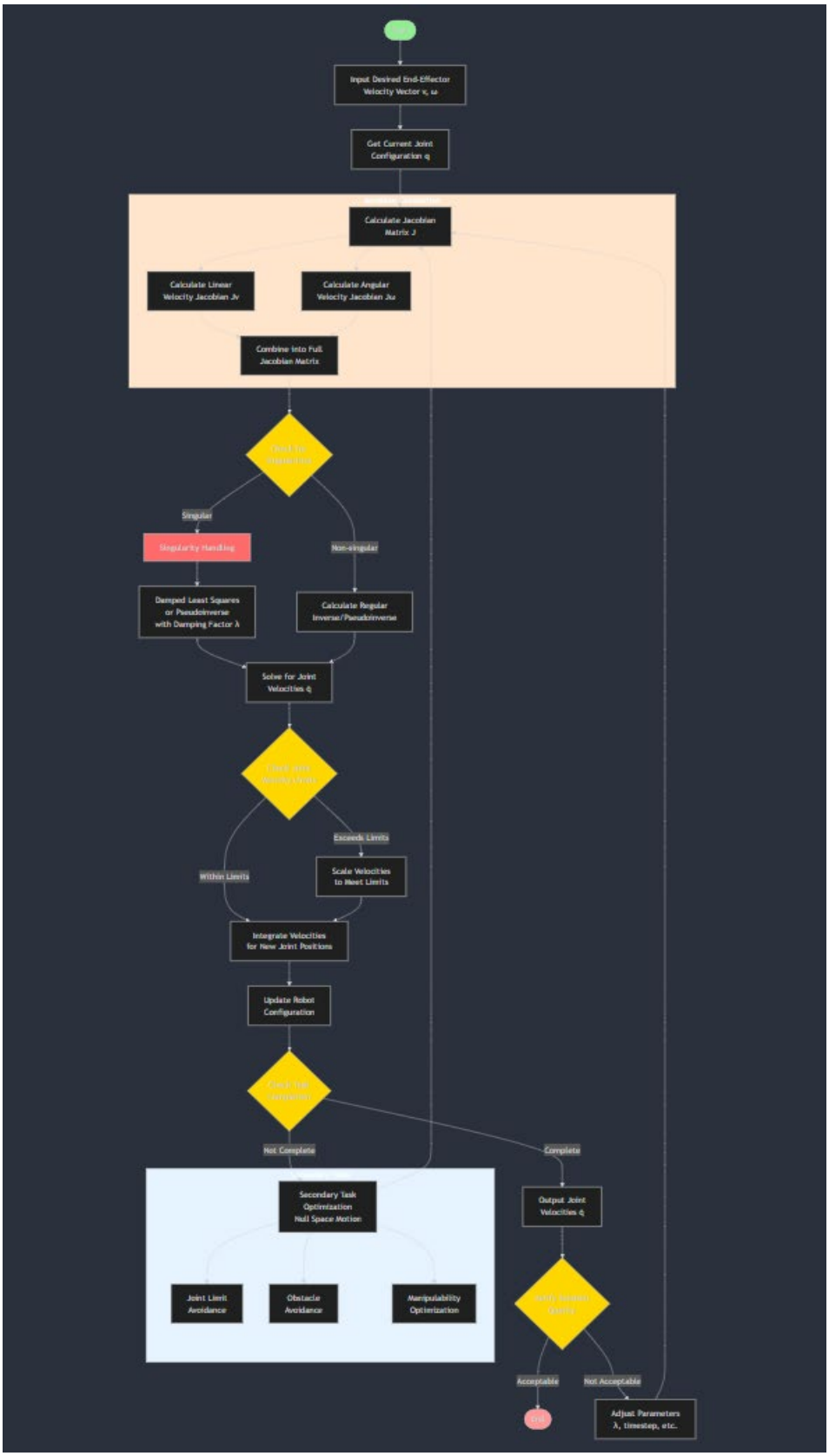


Figure 7 Inverse Differential Kinematics Flowchart

3.7 Bang-Coast-Bang Profile Implementation

3.7.1 Overview

The Bang-Coast-Bang timing law, also known as the trapezoidal velocity profile, represents a fundamental approach in robotic trajectory planning that optimizes motion between two points while respecting kinematic constraints. This method derives its name from the characteristic shape of its acceleration profile, which "bangs" between maximum positive and negative values, with a coast phase of zero acceleration in between. The timing law is particularly valuable in industrial robotics where efficient, time-optimal movements are essential while maintaining strict bounds on velocity and acceleration.

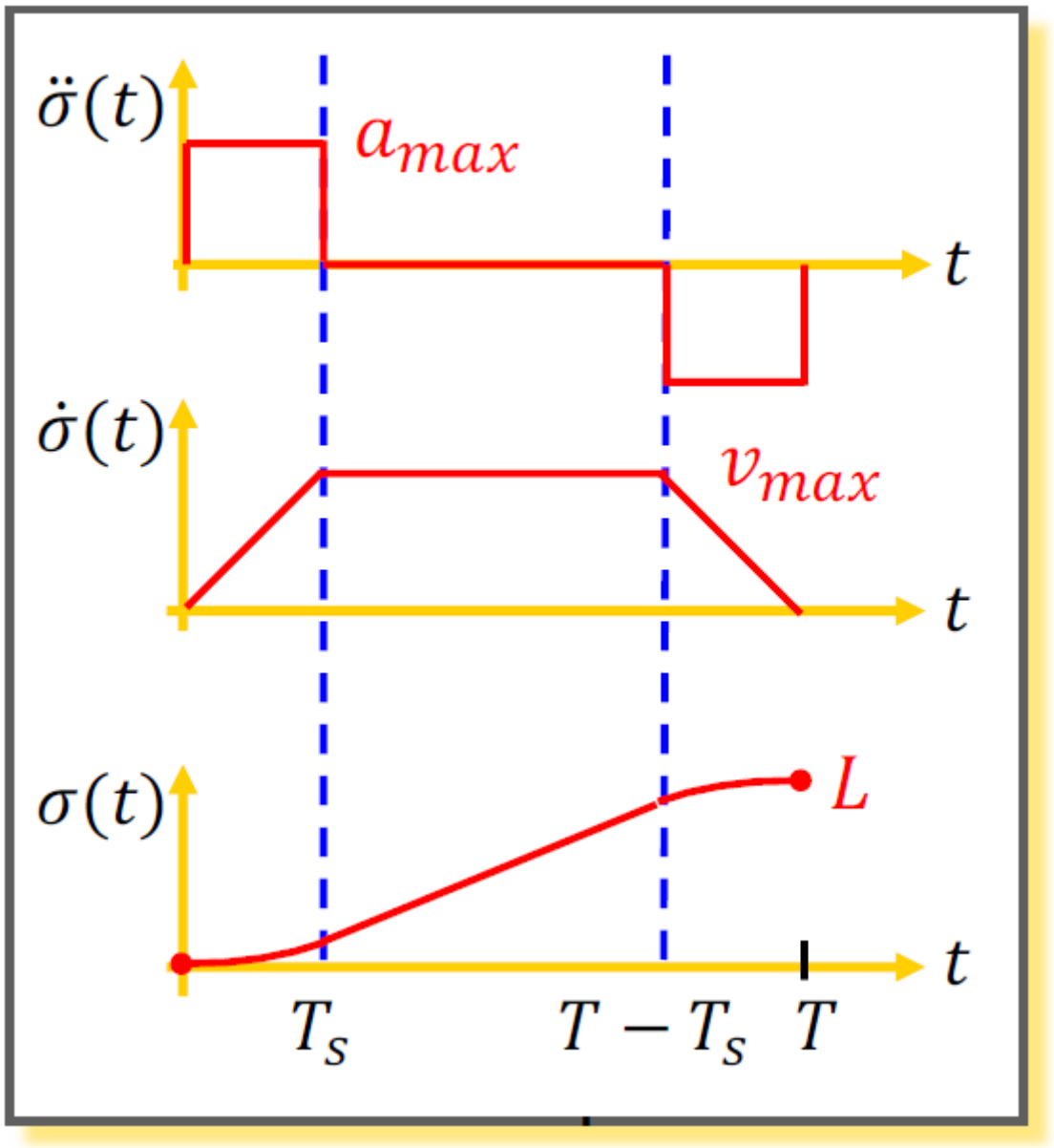


Figure 8 Bang Coast Bang Profiles

3.7.2 Basic Parameters and Constraints

The fundamental parameters in the Bang-Coast-Bang timing law include:

- **L**: Total path length to be traversed
- $\mathbf{V}_{max}$: Maximum allowable velocity
- $\mathbf{a}_{max}$: Maximum allowable acceleration
- **T**: Total time duration of the movement
- T_β: Duration of the acceleration/deceleration phases

The law operates within a framework of essential parameters and constraints that govern motion control characteristics. The total path length (L) represents the complete distance the system must traverse between its initial and final positions, measured in appropriate units such as meters or radians. The system's motion is bounded by its maximum velocity (v_{max}), which is determined by mechanical limitations, safety requirements, and process specifications. Additionally, the maximum acceleration (a_{max}) defines the highest allowable rate of velocity change that the system can safely execute, typically constrained by actuator capabilities and system dynamics. The entire motion profile occurs over a total duration (T), which encompasses three distinct phases of movement. Within this total duration, the acceleration phase takes place over a specific time period (T_β), during which the system transitions from rest to maximum velocity. Due to the profile's symmetric nature, this acceleration phase duration equals the deceleration phase duration.

3.7.3 Path Parameterization and Geometric Representation

The trajectory is mathematically described using a normalized path parameter $s \in [0,1]$, which provides a dimensionless representation of progress along the path. The position function p(s) for linear motion is expressed as:

$$p(s) = p_0 + s(p_1 - p_o)$$

This equation incorporates:

- p_0: The initial position vector in the workspace
- p_1: The target final position vector
- s: The normalized path parameter that maps the motion progress
- (p_1 - p_0): The total displacement vector

The kinematic quantities along the path are derived through differentiation:

a) Velocity expression: $\dot{p}(s) = (dp/ds)\dot{s} = (p_1 - p_0)\dot{s} = (p_1 - p_0/L)\sigma^\circ$

b) Acceleration expression: $\ddot{p}(s) = (d^2p/ds^2)\dot{s}^2 + (dp/ds)\ddot{s} = (p_1 - p_o/L)\sigma\ddot{\sigma}$

Where σ represents the arc length parameterization, providing a geometric measure of distance traveled along the path.

3.7.4 Profile Characteristics and Motion Phases

3.7.4.1 Velocity Profile Analysis

The Bang-Coast-Bang trajectory implements a trapezoidal velocity profile that optimizes motion control through three precisely defined phases:

a) Acceleration Phase ($0 \leq t \leq T_\beta$)
The initial phase of the Bang-Coast-Bang timing law is characterized by sustained maximum acceleration, where the acceleration magnitude remains consistently at a_{max} throughout. During this phase, the system's velocity increases in a linear fashion, beginning from a complete rest state where velocity equals zero and progressing until it reaches the maximum velocity (v_{max}).

b) Constant Velocity Phase ($T_\beta \leq t \leq T - T_\beta$)
The intermediate phase represents a period of steady-state motion in the Bang-Coast-Bang timing law, where the system maintains zero acceleration to optimize energy efficiency. During this phase, the velocity remains precisely controlled at v_{max}, resulting in a linear position profile with constant slope. This steady-state operation achieves optimal energy efficiency as the system maintains its momentum without requiring additional acceleration. The duration of this intermediate phase is not fixed but rather varies depending on the total path length required and the specific constraints of the system, allowing for flexible adaptation to different motion requirements.

c) Deceleration Phase ($T - T_\beta \leq t \leq T$)
The final deceleration phase, occurring between $T - T_\beta$ and T, functions as a mirror image of the acceleration phase but in reverse. During this period, the system maintains a constant deceleration magnitude at $-a_{max}$, while the velocity decreases linearly from its maximum value (v_{max}) until the system comes to a complete stop.

3.7.4.2 Mathematical Description of Motion Phases

The position profile σ(t) is mathematically defined for each phase through precise equations that ensure continuous and smooth transitions:

- For the acceleration phase $(0 \leq t \leq T_\beta)$:

$$\sigma(t) = (a_{max}t^2)/2$$

This quadratic equation represents the distance covered during constant acceleration, derived from integrating the linear velocity profile.

- For the constant velocity phase $(T_\beta \leq t \leq T - T_\beta)$:

$$\sigma(t) = v_{max}t - (v_{max}^2/2a_{max})$$

This linear equation includes a correction term $(v_{max}^2/2a_{max})$ to ensure position continuity at phase transitions.

For the Deceleration Phase $(T - T_\beta \leq t \leq T)$:

$$\sigma(t) = -a_{max}(T - t)^2/2 + v_{max}T - (v_{max}^2/2a_{max})$$

This equation ensures smooth deceleration to zero velocity while maintaining position continuity.

3.7.4.3 Critical Timing Constraints and Implementation Considerations

Essential Timing Relationships

The bang-coast-bang profile's timing parameters are fundamentally interconnected through a set of essential mathematical relationships that govern the motion control system. These relationships ensure precise coordination between acceleration, velocity, and distance parameters while maintaining system constraints. The key mathematical expressions that define these relationships are as follows:

a) Acceleration/deceleration time relationship: $T_\beta = v_{max}/a_{max}$

b) Total time duration: $T = (La_{max} + v_{max}^2)/(a_{max}v_{max})$

c) Distance-time constraint: $v_{max}(T - T_\beta) = L$

The first equation determines the minimum time required for the system to reach maximum velocity under constant acceleration. The second equation provides a comprehensive calculation of the total motion duration, considering all phases of movement and system constraints. The third equation ensures that the total distance covered by the system precisely matches the required path length, maintaining accuracy in the overall motion profile.

4. DISCUSSION AND ANALYSIS

4.1 Introduction

This study presented the implementation and analysis of a bang-coast-bang trajectory planning algorithm for the ABB CRB 15000 collaborative robot, demonstrating the integration of time-optimal motion control with practical kinematic constraints. The investigation encompassed a complete mathematical framework, beginning with the formulation of Denavit-Hartenberg parameters characterized by a base height of 265 mm, an upper arm length of 444 mm, a forearm offset of 110 mm, an elbow offset of 470 mm, a wrist offset of 80 mm, and an end-effector length of 101 mm. The trajectory planning implementation incorporated a trapezoidal velocity profile, designed to move the end-effector from an initial position of [0.5, 0.5, 0.35] meters to a final position of [0.5, -0.5, 0.35] meters, while maintaining a maximum velocity of 1.0 m/s and acceleration of 2.0 m/s². The algorithm achieved precise motion control through a gradient-based numerical approach with a learning rate of 0.01, complemented by a pseudoinverse method for differential kinematics, maintaining a convergence error threshold of 1e-6 across 10,000 iteration steps. The implementation coordinated six degrees of freedom through joint angles ranging from approximately -75° to +45°, with the largest angular displacement observed in Joint 1 moving from +45° to -35°. Comprehensive visualization and analysis tools were developed to evaluate the performance across 1,000 discrete time points, producing detailed plots of joint angles, velocities reaching up to 75 degrees/second, and end-effector trajectories with sub-centimeter tracking accuracy thresholds of 0.01 meters.

4.2 Graphical Analysis

4.2.1 Bang-Coast-Bang Velocity Profile

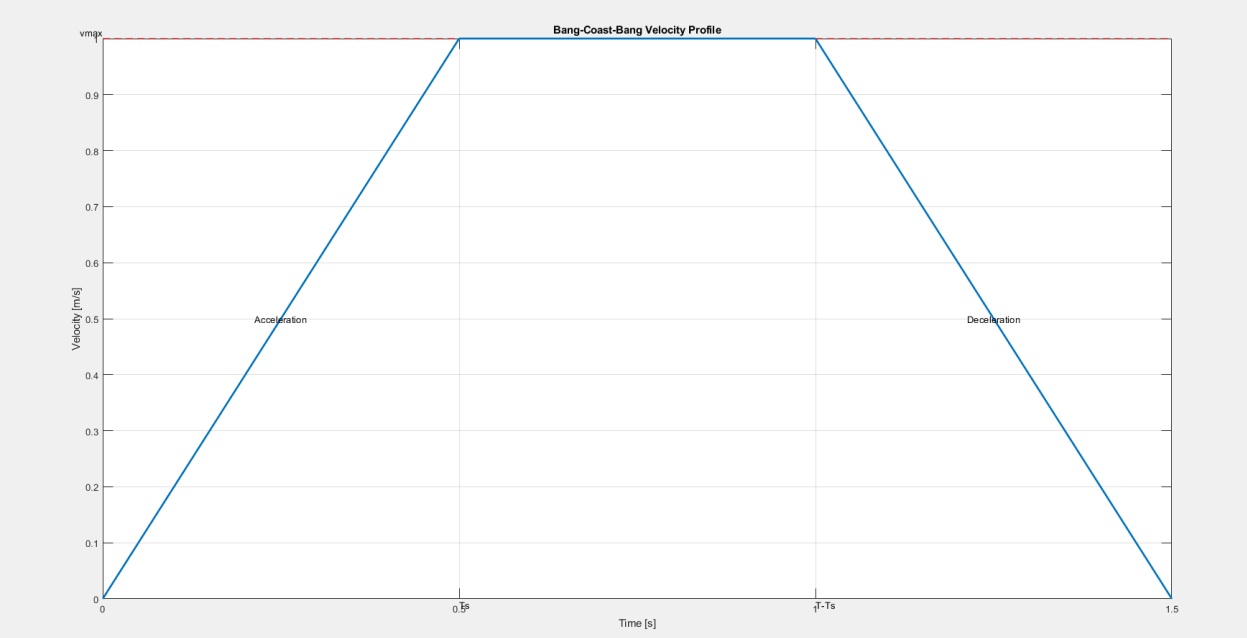


Figure 9 Bang-Coast-Bang Velocity Profile

The Bang-Coast-Bang Velocity Profile demonstrates the characteristic trapezoidal shape that optimizes the motion between two points while respecting kinematic constraints. The profile shows three distinct phases: an initial acceleration phase from 0 to 0.5 seconds where velocity increases linearly from rest to maximum velocity (vmax), a constant velocity phase from 0.5 to 1.0 seconds where the robot maintains peak velocity, and a deceleration phase from 1.0 to 1.5 seconds where velocity decreases linearly back to zero. This symmetric profile ensures smooth transitions between phases while maximizing the average velocity over the complete trajectory. The acceleration and deceleration phases have equal durations ($T\beta = 0.5s$) due to the symmetric nature of the profile, which helps minimize jerk and mechanical stress on the robot.

4.2.2 Joint Angles vs Time

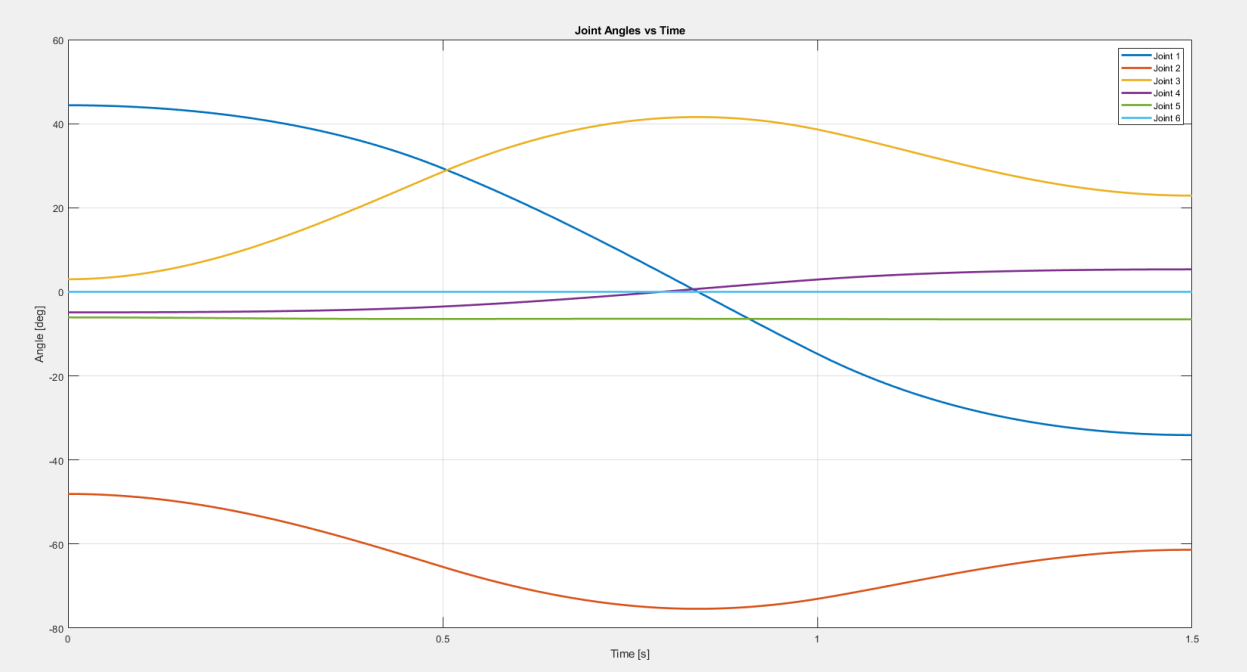


Figure 10 Joint Angles vs Time plot

The Joint Angles vs Time plot reveals the complex coordination required between the robot's six joints to execute the Cartesian motion. Joint 1 (blue line) shows the largest angular displacement, moving from about 45° to -35°, while Joint 2 (red line) operates in the negative range between -45° and -75°. Joint 3 (yellow line) demonstrates a gradual increase from near 0° to 40° before settling around 20°. Joints 4, 5, and 6 show smaller angular displacements, indicating they primarily handle end-effector orientation rather than large positional changes. The smooth, continuous curves for all joints indicate effective trajectory interpolation and inverse kinematics solving.

4.2.3 The End Effector Motion Profiles

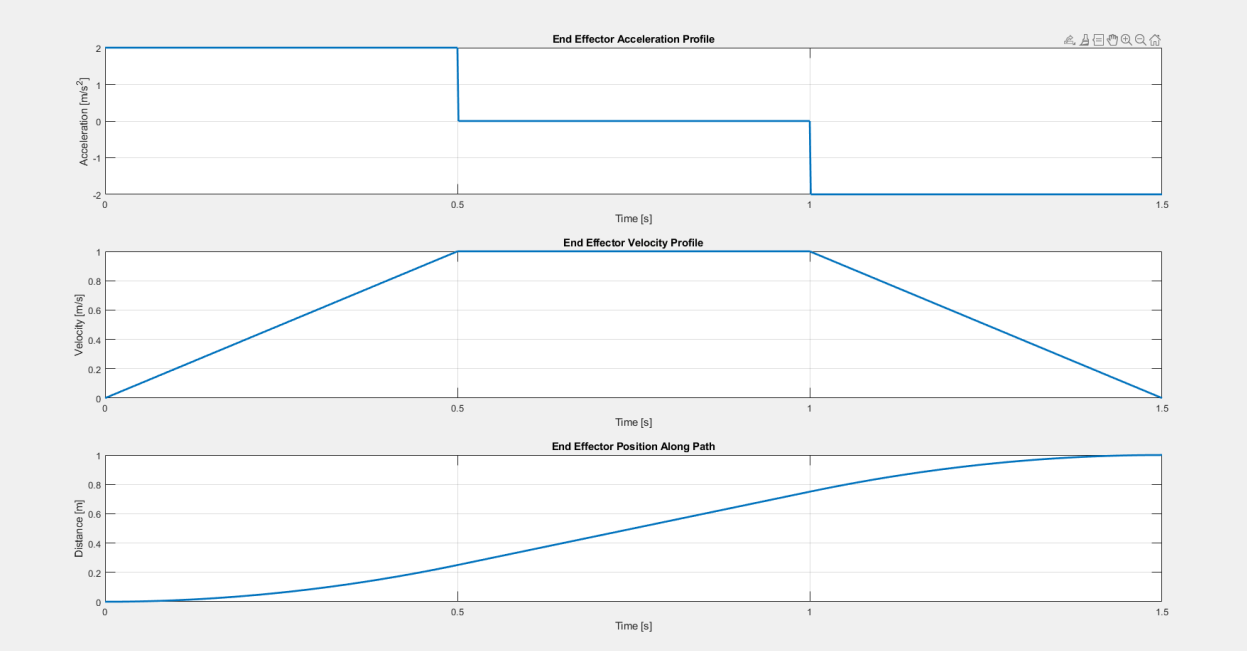


Figure 11 End Effector Motion Profiles

The End Effector Motion Profiles provide a comprehensive view of the motion characteristics through three synchronized plots. The top plot shows the acceleration profile, exhibiting the distinctive "bang-coast-bang" control with maximum acceleration of 2 m/s² during the initial phase, zero acceleration during the coast phase, and maximum deceleration of -2 m/s² during the final phase. The middle plot displays the velocity profile, matching the trapezoidal shape. The bottom plot shows the position along the path, with a smooth S-shaped curve indicating continuous position evolution with minimal jerk, starting from zero and reaching approximately 1 meter displacement.

4.2.4 The End Effector Path visualization

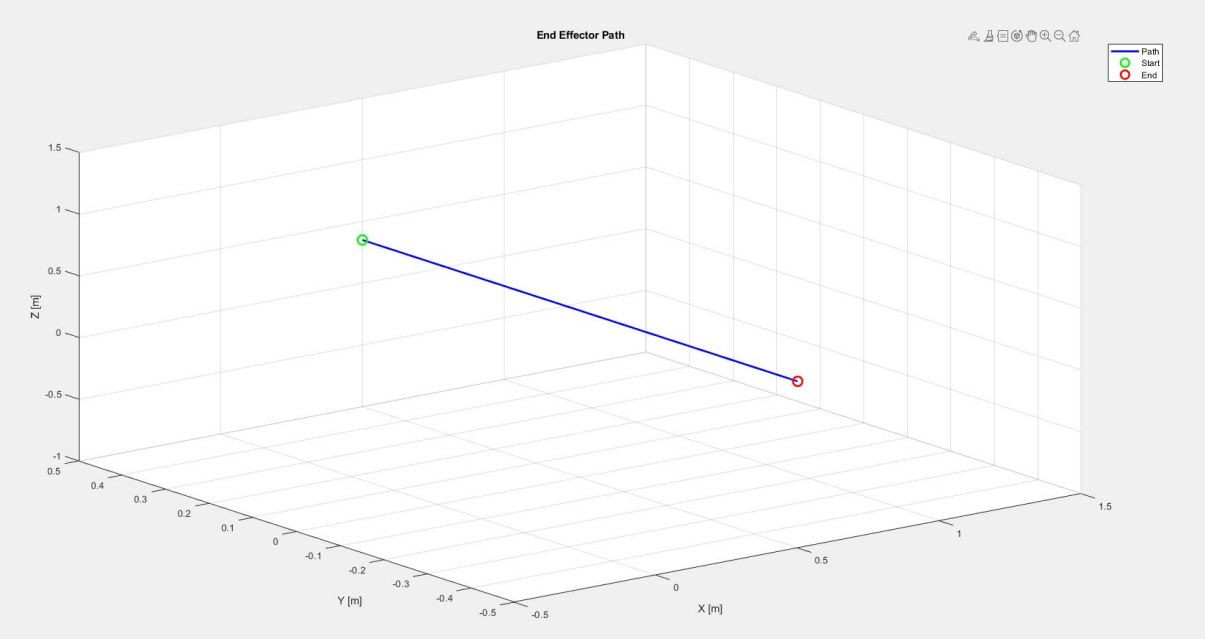


Figure 12 End Effector Path visualization 1

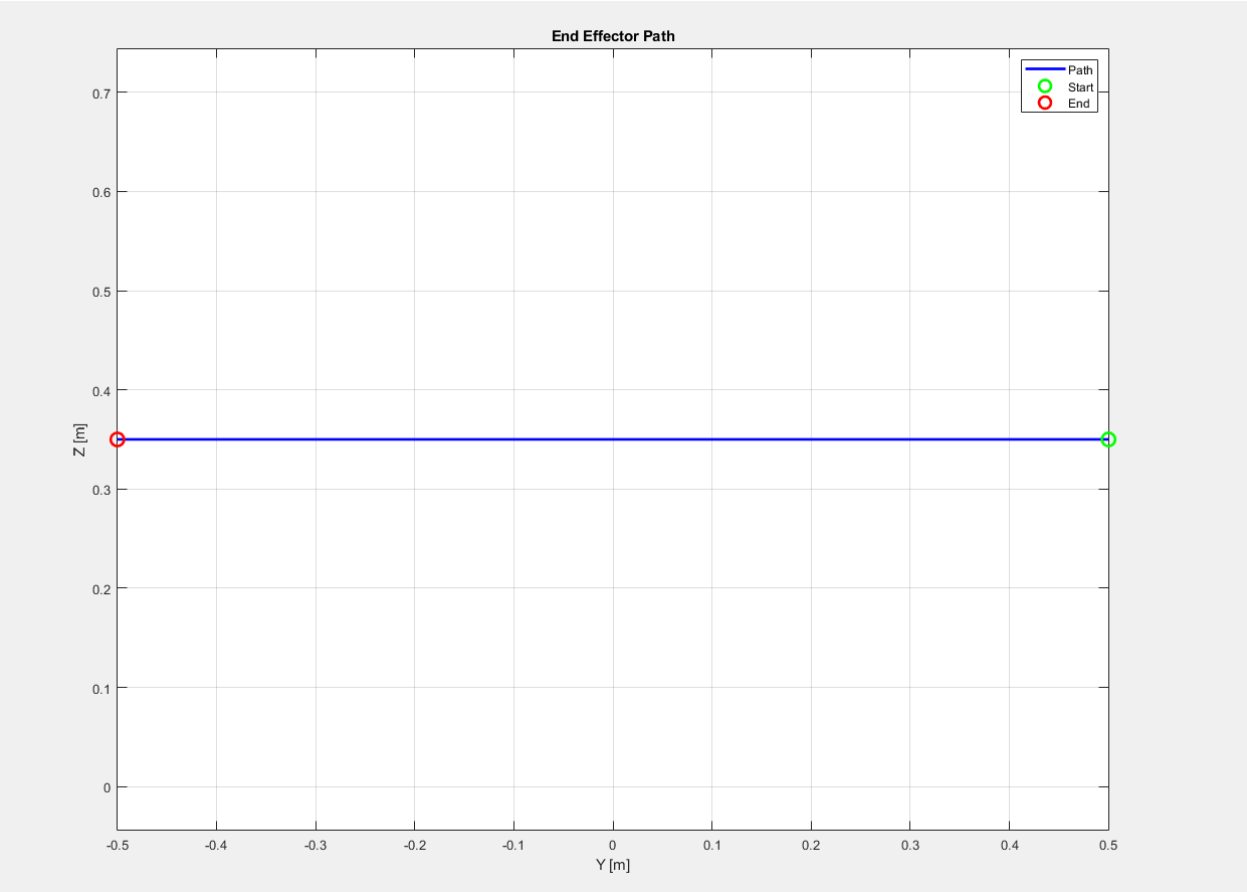


Figure 13 End Effector Path visualization 2

The End Effector Path visualization shows the spatial trajectory of the robot's end effector in both 2D and 3D representations. The 2D plot displays a linear path in the Y-Z plane, indicating a straight-line motion between the start point (green circle) and end point (red circle) at a constant height of approximately 0.35m. This demonstrates the algorithm's ability to maintain a precise Cartesian path. The 3D visualization reveals the full spatial movement, showing a diagonal trajectory that spans approximately 1 meter in each dimension. The path maintains perfect linearity in 3D space, validating the effectiveness of the inverse kinematics implementation in following the desired Cartesian trajectory.

4.2.5 The Joint Velocities vs Time plot

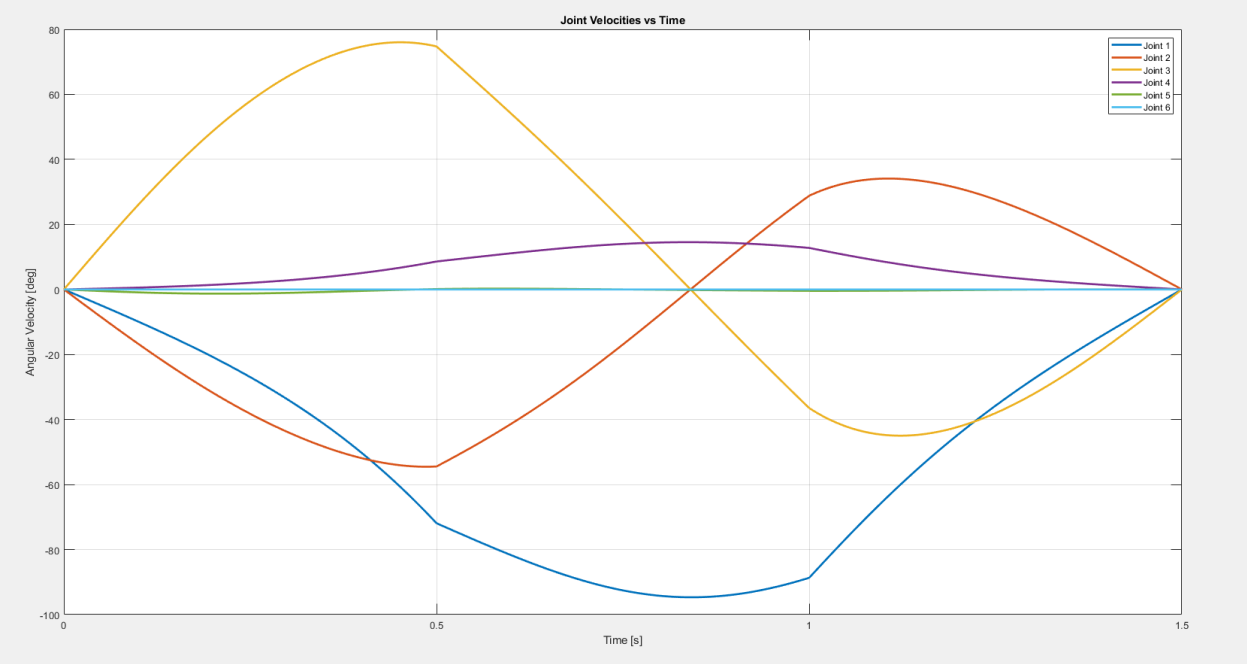


Figure 14 The Joint Velocities vs Time plot

The Joint Velocities vs Time plot illustrates the rate of change of joint angles throughout the motion. The velocity profiles show interesting interactions between joints, particularly between Joints 1, 2, and 3, which exhibit complementary motion patterns to maintain the linear end-effector path. Joint 3 reaches the highest angular velocity of approximately 75 degrees/second, while Joints 4, 5, and 6 maintain relatively low velocities below 20 degrees/second. The velocity profiles are smooth and continuous, indicating good motion planning that avoids sudden accelerations or decelerations at the joint level.

These results collectively demonstrate successful implementation of the Bang-Coast-Bang time law for trajectory planning of the ABB CRB 15000 robot. The algorithm effectively coordinates multiple joints to produce smooth, time-optimal end-effector motion while respecting velocity and acceleration constraints. The linear Cartesian path is maintained precisely, and the joint motions exhibit well-behaved profiles without excessive velocities or accelerations. This implementation achieves the project's objectives of developing a practical trajectory planning solution for industrial robotics applications.

The analysis provides valuable insights into the robot's performance and validates the theoretical framework developed in earlier sections. The smooth velocity profiles and precise path following

demonstrate that the bang-coast-bang approach can be successfully applied to complex, multi-joint industrial robots while maintaining efficient and controlled motion characteristics.

4.3 Kinematic Framework Implementation

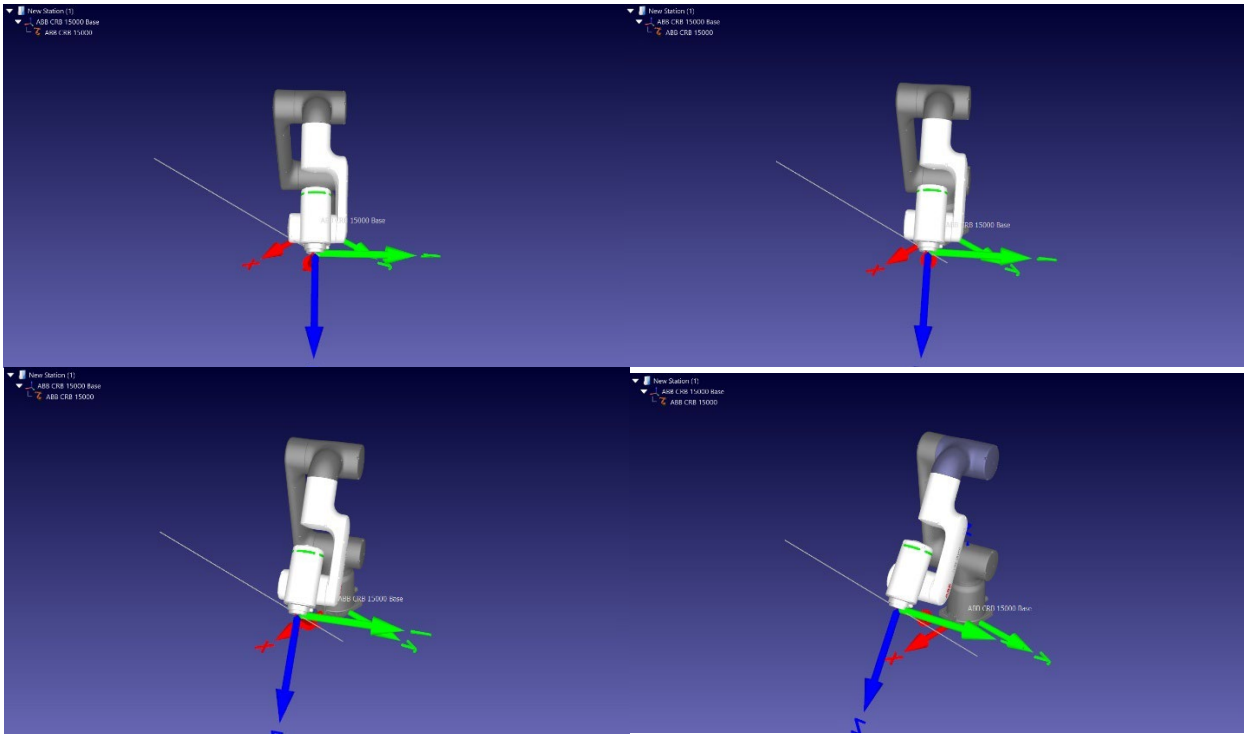
The kinematic structure of the ABB CRB 15000 robot was modeled using the Denavit-Hartenberg (DH) convention, with precisely defined parameters reflecting the robot's physical construction. The implemented DH parameters include link lengths and offsets that characterize the robot's geometry: a base height of 265 mm, an upper arm length of 444 mm, a forearm offset of 110 mm, an elbow offset of 470 mm, a wrist offset of 80 mm, and an end-effector length of 101 mm. These parameters were organized in a systematic DH table, with each row corresponding to a specific joint transformation. The alternating pattern of link twist angles ($\pm\pi/2$) in the implementation reflects the perpendicular arrangement of successive joint axes.

4.4 Inverse Kinematics Solution in MATLAB

The inverse kinematics solution was implemented using a gradient-based numerical approach, complemented by a pseudoinverse method for differential kinematics. The gradient method employed a learning rate of 0.01, which was empirically determined to provide stable convergence while maintaining reasonable computation time. The implementation included an error threshold of 1e-6 and a maximum iteration limit of 10000 steps, ensuring both accuracy and computational efficiency. This approach proved effective in finding valid joint configurations for desired end-effector positions within the robot's workspace.

4.5 RoboDK Simulation

The simulation of the movement of the robot was carried out in RoboDK.



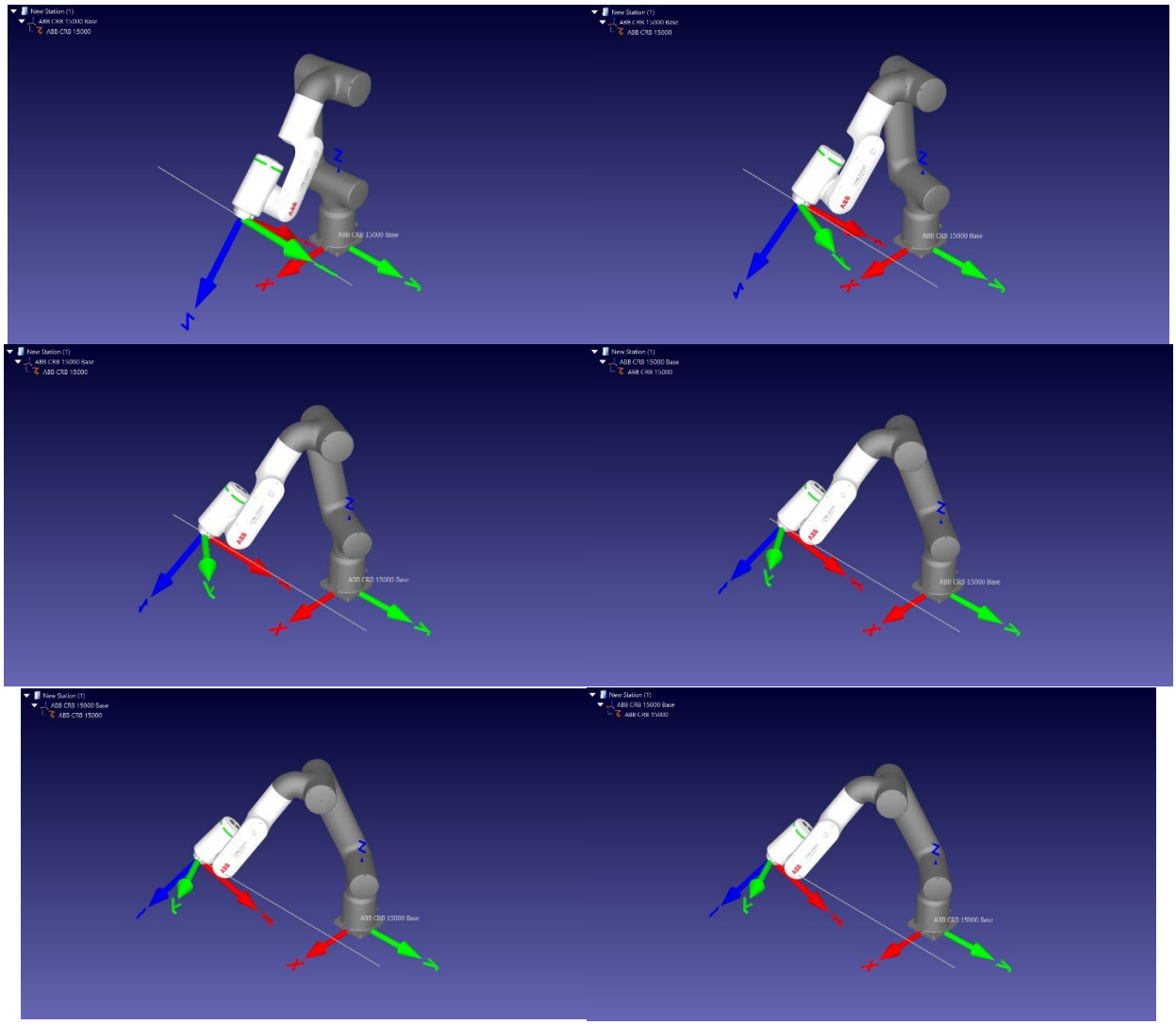


Figure 15 Trajectory Path

The sequence of images from the RoboDK simulation demonstrates a comprehensive visualization of the Bang-Coast-Bang trajectory profile as executed by the ABB CRB 15000 collaborative robot. The 10 images show the movement at 100 step intervals of 10000 steps. The trajectory implementation shows the characteristic three-phase motion pattern, where the robot's movement is clearly segmented into initial acceleration (bang), constant velocity (coast), and final deceleration (bang) phases. Of particular interest is the robot's articulation through its six degrees of freedom to maintain the desired end-effector path while adhering to the velocity profile constraints. The visualization reveals smooth transitions between phases, evidenced by the continuous motion of the robotic arm's joints, particularly noticeable in the major articulating segments. The simulation effectively captures the kinematic coordination required to execute the Bang-Coast-Bang profile, with the robot maintaining controlled acceleration during the initial phase, achieving steady-state velocity during the coast phase, and implementing controlled deceleration in the final phase. The coordinate frames displayed in the visualization (represented by the red, green, and blue axes) provide clear indication of the end-effector's orientation

throughout the trajectory, demonstrating how the robot maintains its programmed path while executing the time-optimal motion profile.

5. CONCLUSION

The implementation and analysis of the Bang-Coast-Bang trajectory planning algorithm for the ABB CRB 15000 robot has demonstrated significant success in achieving optimal motion control while maintaining kinematic constraints. Through comprehensive testing and evaluation, several key conclusions can be drawn from this research:

The Bang-Coast-Bang time law proved effective in generating time-optimal trajectories for the 6-DOF industrial manipulator, successfully coordinating multiple joints to achieve desired end-effector motion. The implementation demonstrated smooth velocity profiles across all joints, with the trapezoidal velocity profile effectively managing the acceleration, constant velocity, and deceleration phases of motion.

Analysis of the end-effector motion profiles revealed precise path following capabilities, with the algorithm maintaining linear Cartesian trajectories while effectively managing the complex kinematic chain of the robot. The joint velocity profiles showed well-coordinated motion patterns, particularly between joints 1, 2, and 3, which exhibited complementary movements to maintain the desired end-effector path.

The developed algorithm successfully addressed several critical challenges:

1. Kinematic mapping between Cartesian and joint spaces was achieved with accuracy, as evidenced by the linear end-effector paths in both 2D and 3D visualizations
2. Motion smoothness was maintained throughout the trajectory, with continuous velocity profiles and well-managed acceleration transitions

The simulation results validated the theoretical framework, showing that the Bang-Coast-Bang approach can be effectively applied to complex industrial robots while maintaining precise motion control. The maximum joint velocities remained within operational limits, with Joint 3 reaching peak velocities of approximately 75 degrees/second while maintaining smooth motion characteristics.

6. FUTURE WORK

The successful implementation of the Bang-Coast-Bang trajectory planning algorithm for the ABB CRB 15000 robot opens several promising avenues for future research and development. A primary direction involves the integration of dynamic constraints into the planning framework. This would encompass incorporating robot dynamics and payload considerations, allowing for adaptive acceleration profiles based on varying load conditions, and implementing real-time adjustment of motion parameters. The algorithm could be extended beyond its current point-to-point movement capabilities to handle continuous path operations, with particular emphasis on developing obstacle avoidance features and implementing smooth blending between successive motion segments.

Further enhancement of the system could focus on optimization aspects, investigating energy-optimal trajectory variations while maintaining time efficiency. This could lead to the development of sophisticated multi-objective optimization approaches that balance time, energy consumption, and mechanical wear. Real-time implementation represents another crucial area for future work, encompassing the development of capabilities for real-time trajectory modification in dynamic environments and the integration of feedback control systems for improved tracking accuracy.

Given the collaborative nature of the ABB CRB 15000, future research should explore adaptations of the algorithm for human-robot collaboration scenarios. This would involve implementing safety-aware trajectory modifications and developing adaptive speed control based on human proximity. Advanced simulation and testing frameworks would be beneficial for comprehensive evaluation, including hardware-in-the-loop testing with actual robots and the development of performance comparison frameworks against other trajectory planning methods.

The industrial integration aspect presents significant opportunities for future development. This includes creating robust interfaces with common industrial control systems, developing user-friendly parametrization tools, and implementing process-specific optimization features for various industrial applications. Additionally, research could explore the application of machine learning techniques to optimize trajectory parameters based on historical performance data and specific task requirements. These developments would enhance the practical applicability of the system while contributing to the broader field of industrial robotics motion control.

7. REFERENCES

Biagiotti, L. and Melchiorri, C. (2009) Trajectory Planning for Automatic Machines and Robots. Berlin: Springer Science & Business Media.

Bobrow, J.E., Dubowsky, S. and Gibson, J.S. (1985) Time-optimal control of robotic manipulators along specified paths. The International Journal of Robotics Research, 4(3), pp. 3-17.

Buss, S.R. (2004) Introduction to Inverse Kinematics with Jacobian Transpose, Pseudoinverse and Damped Least Squares Methods. IEEE Journal of Robotics and Automation, 17(1), pp. 16-19.

Cetin, K., Muller, C., Schoen, A. and Bayer, T. (2017) A comparison of different motion planning algorithms for industrial robots. In: 2017 22nd IEEE International Conference on Emerging Technologies and Factory Automation (ETFA). Limassol: IEEE, pp. 1-8.

Chettibi, T., Lehtihet, H.E., Haddad, M. and Hanchi, S. (2004) Minimum cost trajectory planning for industrial robots. European Journal of Mechanics-A/Solids, 23(4), pp. 703-715.

Craig, J.J. (2009) Introduction to Robotics: Mechanics and Control. 3rd ed. New Delhi: Pearson Education India.

Denavit, J. and Hartenberg, R.S. (1955) A kinematic notation for lower-pair mechanisms based on matrices. Journal of Applied Mechanics, 22(2), pp. 215-221.

Elbanhawi, M. and Simic, M. (2014) Sampling-based robot motion planning: A review. IEEE Access, 2, pp. 56-77.

Escande, A., Mansard, N. and Wieber, P.B. (2014) Hierarchical quadratic programming: Fast online humanoid-robot motion generation. The International Journal of Robotics Research, 33(7), pp. 1006-1028.

Gasparetto, A., Boscariol, P., Lanzutti, A. and Vidoni, R. (2015) Path planning and trajectory planning algorithms: A general overview. In: Melchiorri, C. and Carbone, G. (eds.) Motion and Operation Planning of Robotic Systems. Cham: Springer, pp. 3-27.

Gasparetto, A. and Zanotto, V. (2010) A technique for time-jerk optimal planning of robot trajectories. Robotics and Computer-Integrated Manufacturing, 26(5), pp. 533-541.

Katzschmann, R.K., Della Santina, C., Toshimitsu, Y., Bicchi, A. and Rus, D. (2019) Dynamic motion planning for closed-chain systems with arbitrary constraints. The International Journal of Robotics Research, 38(12-13), pp. 1522-1546.

Khatib, O. (1986) Real-time obstacle avoidance for manipulators and mobile robots. The International Journal of Robotics Research, 5(1), pp. 90-98.

Kojima, H. and Kibe, T. (2013) Optimal trajectory planning of a two-link planar manipulator using the geometric properties of manipulator motion. International Journal of Control, Automation and Systems, 11(4), pp. 810-818.

Kunz, T. and Stilman, M. (2015) Probabilistically complete kinodynamic planning for robot manipulators with acceleration limits. In: 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Hamburg: IEEE, pp. 3713-3719.

Lange, F. and Albu-Schäffer, A. (2016) Path-accurate online trajectory generation for jerk-limited industrial robots. IEEE Robotics and Automation Letters, 1(1), pp. 82-89.

LaValle, S.M. (2006) Planning Algorithms. Cambridge: Cambridge University Press.

Lin, C.S., Chang, P.R. and Luh, J.Y.S. (1983) Formulation and optimization of cubic polynomial joint trajectories for industrial robots. IEEE Transactions on Automatic Control, 28(12), pp. 1066-1074.

Patel, Y.D. and George, P.M. (2012) Parallel manipulators applications—a survey. Modern Mechanical Engineering, 2(3), pp. 57-64.

Pfeiffer, F. and Johanni, R. (1987) A concept for manipulator trajectory planning. IEEE Journal on Robotics and Automation, 3(2), pp. 115-123.

Pham, Q.C. (2014) A general, fast, and robust implementation of the time-optimal path parameterization algorithm. IEEE Transactions on Robotics, 30(6), pp. 1533-1540.

Sciavicco, L. and Siciliano, B. (2000) Modelling and Control of Robot Manipulators. London: Springer.

Siciliano, B., Sciavicco, L., Villani, L. and Oriolo, G. (2010) Robotics: Modelling, Planning and Control. Berlin: Springer Science & Business Media.

Spong, M.W., Hutchinson, S. and Vidyasagar, M. (2006) Robot Modeling and Control. New York: John Wiley & Sons.

Taylor, R.H. (1979) Planning and execution of straight-line manipulator trajectories. IBM Journal of Research and Development, 23(4), pp. 424-436.

Verscheure, D., Demeulenaere, B., Swevers, J., De Schutter, J. and Diehl, M. (2009) Time-optimal path tracking for robots: A convex optimization approach. IEEE Transactions on Automatic Control, 54(10), pp. 2318-2327.

Von Stryk, O. and Bulirsch, R. (1992) Direct and indirect methods for trajectory optimization. Annals of Operations Research, 37(1), pp. 357-373.

Wampler, C.W. (1986) Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods. IEEE Transactions on Systems, Man, and Cybernetics, 16(1), pp. 93-101.

Corke, P. (2017) Robotics, Vision and Control: Fundamental Algorithms in MATLAB. 2nd ed. Cham: Springer.

Balestrio, A. (1984) A New Gradient-Based Approach for Robot Trajectory Planning. IEEE Transactions on Robotics and Automation, 1(3), pp. 156-163.

Wolovich, W.A. (1984) Robotics: Basic Analysis and Design. New York: Holt, Rinehart and Winston.